PATTERN RECOGNITION LAB

Brain Tumor MRI Classification

Q1–Q15 / Kaggle Markdown and Python cells

Add the Sartaj MRI dataset via Add Input. Copy each Markdown cell and Python cell into Kaggle in order. This lab uses 600 images from the Training folder, with an expected split of 480 training, 60 validation and 60 test images. The split is at image level.

Enable a GPU for training. Q10 may require internet access to download ImageNet weights. Test results will be produced when you run the notebook.

Q01 Load the dataset and count images

MARKDOWN CELL

```
## Q1. Load the dataset and count images

Read the three tumor folders, report source counts, and select 200 images per class using
seed 42.
```

PYTHON CELL

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from PIL import Image
from IPython.display import display

SEED = 42
tf.keras.utils.set_random_seed(SEED)
folders = ["glioma_tumor", "meningioma_tumor", "pituitary_tumor"]
class_names = ["Glioma", "Meningioma", "Pituitary"]
root = next((p for p in Path("/kaggle/input").rglob("Training")
             if all((p / name).is_dir() for name in folders)), None)
if root is None:
    raise FileNotFoundError("Add the Sartaj MRI dataset in Kaggle.")

records = []
for label, folder in enumerate(folders):
    for path in sorted((root / folder).iterdir()):
        if path.is_file() and path.suffix.lower() in {".jpg", ".jpeg", ".png"}:
            records.append((path, label, class_names[label]))

source = pd.DataFrame(records, columns=["path", "label", "class_name"])
print("Classes:", class_names)
print("Total source images in the three classes:", len(source))
display(source["class_name"].value_counts())
dataset = (source.groupby("label", group_keys=False)
           .sample(n=200, random_state=SEED).reset_index(drop=True))
print("Selected images:", len(dataset))
display(dataset["class_name"].value_counts())
```

Q02 Display five random MRI images

MARKDOWN CELL

```
## Q2. Display five random MRI images

Sample five images and place the corresponding class name above each image.
```

PYTHON CELL

```
samples = dataset.sample(n=5, random_state=SEED)
fig, axes = plt.subplots(1, 5, figsize=(15, 4))
for ax, row in zip(axes, samples.itertuples(index=False)):
    with Image.open(row.path) as image:
        ax.imshow(np.asarray(image.convert("RGB")))
    ax.set_title(row.class_name)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Q03 Inspect original dimensions

MARKDOWN CELL

```
## Q3. Inspect original dimensions

Report the original height, width and number of channels for the five sampled images.
```

PYTHON CELL

```
rows = []
for row in samples.itertuples(index=False):
    with Image.open(row.path) as image:
        width, height = image.size
        rows.append([row.path.name, row.class_name, height,
                     width, len(image.getbands())])
display(pd.DataFrame(rows, columns=[
    "Image", "Class", "Height", "Width", "Channels"
]))
```

Q04 Preprocess MRI images

MARKDOWN CELL

```
## Q4. Preprocess MRI images

Convert to RGB, resize to 224 x 224 pixels and scale pixel values to [0, 1]. Display one
processed image.
```

PYTHON CELL

```
X = np.empty((len(dataset), 224, 224, 3), dtype=np.float32)
for i, path in enumerate(dataset["path"]):
    with Image.open(path) as image:
        resized = image.convert("RGB").resize(
            (224, 224), Image.Resampling.BILINEAR
        )
        X[i] = np.asarray(resized, dtype=np.float32) / 255.0
y = dataset["label"].to_numpy(dtype=np.int32)
print("Shape:", X.shape, "| Pixel range:", X.min(), X.max())
plt.imshow(X[0])
plt.title(class_names[y[0]])
plt.axis("off")
plt.show()
```

Q05 Split into training, validation and test sets

MARKDOWN CELL

```
## Q5. Split into training, validation and test sets

Use stratified sampling and a fixed seed. Split off 20% first, then divide it equally
into validation and test sets.
```

PYTHON CELL

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
)
print("Train:", len(y_train), "Validation:", len(y_val), "Test:", len(y_test))
```

Q06 Verify class distributions

MARKDOWN CELL

```
## Q6. Verify class distributions

Report each class count and its percentage of the corresponding split.
```

PYTHON CELL

```
rows = []
for split, labels in [("Train", y_train), ("Validation", y_val),
                      ("Test", y_test)]:
    counts = np.bincount(labels, minlength=len(class_names))
    for name, count in zip(class_names, counts):
        rows.append([split, name, count, 100 * count / len(labels)])
display(pd.DataFrame(rows, columns=[
    "Split", "Class", "Count", "Percentage (%)"
]).round(2))
```

Q07 Build a custom CNN

MARKDOWN CELL

```
## Q7. Build a custom CNN

Use two convolutional layers, one max-pooling layer, and one fully connected hidden
layer. Softmax produces three class probabilities.
```

PYTHON CELL

```
from tensorflow.keras import layers

cnn_model = tf.keras.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Conv2D(32, 3, padding="same", activation="relu"),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding="same", activation="relu"),
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dense(3, activation="softmax"),
], name="custom_cnn")
cnn_model.summary()
```

Q08 Train the CNN for 10 epochs

MARKDOWN CELL

```
## Q8. Train the CNN for 10 epochs

Use Adam and sparse categorical cross-entropy for integer labels. Report training and
validation accuracy per epoch.
```

PYTHON CELL

```
EPOCHS = 10
BATCH_SIZE = 16

cnn_model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
cnn_history = cnn_model.fit(
    X_train, y_train, validation_data=(X_val, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE
)
display(pd.DataFrame({
    "Epoch": range(1, EPOCHS + 1),
    "Train accuracy (%)": 100 * np.array(cnn_history.history["accuracy"]),
    "Val accuracy (%)": 100 * np.array(cnn_history.history["val_accuracy"]),
}).round(2))
```

Q09 Plot CNN learning curves

MARKDOWN CELL

```
## Q9. Plot CNN learning curves

Show training and validation accuracy in one figure and training and validation loss in a
separate figure.
```

PYTHON CELL

```
for metric in ("accuracy", "loss"):
    epochs = range(1, EPOCHS + 1)
    plt.figure(figsize=(7, 4))
    plt.plot(epochs, cnn_history.history[metric], label="Training")
    plt.plot(epochs, cnn_history.history[f"val_{metric}"], label="Validation")
    plt.title(f"CNN {metric.capitalize()}")
    plt.xlabel("Epoch")
    plt.ylabel(metric.capitalize())
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
```

Q10 Build pretrained ResNet-50

MARKDOWN CELL

```
## Q10. Build pretrained ResNet-50

Freeze the ImageNet backbone and add a three-class softmax head. Restore the [0, 255]
input range before ResNet-specific preprocessing.
```

PYTHON CELL

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

backbone = ResNet50(weights="imagenet", include_top=False,
                    input_shape=(224, 224, 3))
backbone.trainable = False
inputs = layers.Input(shape=(224, 224, 3))
x = layers.Rescaling(255.0)(inputs)
x = layers.Lambda(preprocess_input)(x)
x = backbone(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(3, activation="softmax")(x)
resnet_model = tf.keras.Model(inputs, outputs, name="frozen_resnet50")
resnet_model.summary()
```

Q11 Train ResNet-50 for 10 epochs

MARKDOWN CELL

```
## Q11. Train ResNet-50 for 10 epochs

Train the new classification head with Adam and batch size 16. Use the same training and
validation splits as the CNN.
```

PYTHON CELL

```
resnet_model.compile(optimizer="adam",
                     loss="sparse_categorical_crossentropy",
                     metrics=["accuracy"])
resnet_history = resnet_model.fit(
    X_train, y_train, validation_data=(X_val, y_val),
    epochs=EPOCHS, batch_size=BATCH_SIZE
)
```

Q12 Compare the learning curves

MARKDOWN CELL

```
## Q12. Compare the learning curves

Compare accuracy and loss for both models. Solid lines show training and dashed lines
show validation.
```

PYTHON CELL

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for ax, metric in zip(axes, ("accuracy", "loss")):
    for name, history, color in [
        ("CNN", cnn_history, "tab:blue"),
        ("ResNet-50", resnet_history, "tab:orange"),
    ]:
        epochs = range(1, EPOCHS + 1)
        ax.plot(epochs, history.history[metric], color=color,
                label=f"{name} train")
        ax.plot(epochs, history.history[f"val_{metric}"], "--",
                color=color, label=f"{name} validation")
    ax.set(xlabel="Epoch", ylabel=metric.capitalize(), title=metric.capitalize())
    ax.legend()
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Q13 Evaluate both models on the test set

MARKDOWN CELL

```
## Q13. Evaluate both models on the test set

Compute accuracy, macro precision, macro recall and macro F1. Macro averaging gives equal
weight to each class.
```

PYTHON CELL

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

class_ids = list(range(len(class_names)))
predictions = {}
results = []
for name, model in [("CNN", cnn_model), ("ResNet-50", resnet_model)]:
    scores = model.predict(X_test, batch_size=BATCH_SIZE, verbose=0)
    y_pred = scores.argmax(axis=1)
    predictions[name] = y_pred
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test, y_pred, labels=class_ids, average="macro", zero_division=0
    )
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Macro Precision": precision,
        "Macro Recall": recall,
        "Macro F1": f1,
    })
comparison = pd.DataFrame(results).set_index("Model")
display(comparison.round(4))
```

Q14 Analyze the highest-accuracy model

MARKDOWN CELL

```
## Q14. Analyze the highest-accuracy model

Display the confusion matrix and classification report. Rows are actual classes and
columns are predicted classes. Show both models if tied.
```

PYTHON CELL

```
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

best_accuracy = comparison["Accuracy"].max()
best_models = comparison.index[comparison["Accuracy"].eq(best_accuracy)]
for name in best_models:
    y_pred = predictions[name]
    matrix = confusion_matrix(y_test, y_pred, labels=class_ids)
    ConfusionMatrixDisplay(matrix, display_labels=class_names).plot(
        cmap="Blues", values_format="d", xticks_rotation=30
    )
    plt.title(f"{name} | Test accuracy: {best_accuracy:.2%}")
    plt.tight_layout()
    plt.show()
    print(classification_report(
        y_test, y_pred, labels=class_ids, target_names=class_names,
        digits=4, zero_division=0
    ))
```

Q15 Summarize test performance

MARKDOWN CELL

```
## Q15. Summarize test performance

Present the test metrics as percentages in a table and grouped bar chart. Identify the
model or models with the highest test accuracy.
```

PYTHON CELL

```
comparison_percent = comparison.mul(100)
display(comparison_percent.round(2))
ax = comparison_percent.T.plot(kind="bar", figsize=(9, 5), rot=0)
ax.set(title="CNN vs ResNet-50 on the MRI Test Set",
       ylabel="Score (%)", ylim=(0, 108))
ax.legend(title="Model")
for bars in ax.containers:
    ax.bar_label(bars, fmt="%.1f", padding=3)
plt.tight_layout()
plt.show()
print(f"Highest test accuracy: {best_accuracy:.2%}")
print("Model(s):", ", ".join(best_models))
```